

1 Question (20 points)

Given an array of integers that are greater than zero as a parameter, write a function called **splitIndex** that checks if it is possible to split this array in two (without changing the order of the elements), such that the sum of the two resulting arrays is the same. The method returns the splitting index (starting index of the second array) if it is possible to split the array; -1 otherwise.

```
public static int splitIndex(int[] A)
```

```
A = {5, 2, 3}
```

Returns 1.

```
A = {2, 3, 2, 1, 1, 1, 2, 1, 1}
```

Returns 3.

```
A = {1, 1, 3}
```

Returns -1.

```
A = {1, 1, 3, 1}
```

Returns -1.

2 Question (20 points)

Write a method **prefixSum** to calculate the prefix sum of a set of integers, which are stored in a file. The name of the file is passed as parameter and the method returns the prefix sum of the numbers in the file. The prefix sum of an array is calculated as following: A is the array; P is the prefix summed array

$$P[0] = A[0];$$
$$P[i] = P[i-1] + A[i]$$

```
public static int[] prefixSum(String fileName)
```

```
fileName -> input1.txt
```

```
22
```

```
2
```

```
9
```

```
23
```

```
40
```

```
Returns {22, 24, 33, 56, 96}
```

```
fileName -> input2.txt
```

```
2
```

```
3
```

```
2
```

```
1
```

```
1
```

```
1
```

```
2
```

```
1
```

```
1
```

```
Returns {2, 5, 7, 8, 9, 10, 12, 13, 14}
```

3 Question (15 points)

Let say you have the function **primes**

```
public static int[] primes(int N)
```

which returns all primes $\leq N$. Write a function **printCoprimes** which takes two integers as input M, N and will print the co-prime factors of those two integers. The co-prime factors of two integers are the prime numbers who divide both integers. You must use the **primes** function given above. **DO NOT** write the function **primes**.

```
public static void printCoprimes(int M, int N)
```

```
M = 120, N = 72
```

```
2
```

```
3
```

```
M = 11, N = 121
```

```
11
```

```
M = 2310, N = 30
```

```
2
```

```
3
```

```
5
```

4 Question (15 points)

Let say you have the functions **factorial** and **power**

```
public static int factorial(int N)
public static int power(int x, int y)
```

which calculate $N!$ and x^y respectively. Write a function **mysterySum** which takes input an integer k and will return an array containing the following sum for the first k numbers. You must use the factorial and power functions given above. **DO NOT** write the functions **factorial** and **power**.

$$\frac{1!}{2^1} + \frac{2!}{2^2} + \dots + \frac{N!}{2^N}$$

```
public static double[] mysterySum(int k)
```

k = 1

Returns {0.5}

k = 3

Returns {0.5, 1, 1.75}

k = 5

Returns {0.5, 1, 1.75, 3.25, 7}

5 Question (15 points)

Write a program that gets an integer value `min` and rolls a 6-sided die until it gets four consecutive rolls in a row that have values of `min` or less than `min`. As the program rolls the die it should print each value rolled, and then it should print a message at the end to indicate how many rolls were made. In the first example, notice that the program stops after rolling 3, 2, 1, and 2 consecutively because these are all values of 3 or less than 3.

```
Enter a min value: 3
524613553212
Finished after 12 rolls!
```

```
Enter a min value: 5
1363554
Finished after 7 rolls!
```

6 Question (15 points)

Write a program that will generate a sequence of numbers according to the following algorithm. Your program will take an integer n . If n is even, the program will divide it by 2. If n is odd, you will multiply it by 3 and add 1. You repeat this process until you get 1.

Enter number: 22

22 11 34 17 52 26 13 40 20 10 5 16 8 4 2 1

Enter number: 1024

1024 512 256 128 64 32 16 8 4 2 1